

Especificação técnica — Aplicação de Finanças Pessoais com Firebase

1. Visão do produto

Aplicação web responsiva para controle financeiro pessoal e familiar, permitindo gerenciar:

- Contas bancárias.
- Dinheiro em espécie.
- Cartões de crédito.
- Receitas e despesas.
- Transferências.
- Orçamentos mensais.
- Contas recorrentes.
- Parcelamentos.
- Metas financeiras.
- Dívidas.
- Assinaturas.
- Patrimônio.
- Investimentos.
- Relatórios.
- Importação de extratos.
- Compartilhamento financeiro entre casal ou família.

A aplicação será construída utilizando serviços gerenciados do Firebase, sem um servidor backend tradicional durante as primeiras fases.

2. Stack tecnológica

2.1 Frontend

- React.
- TypeScript.
- Vite.
- React Router.
- TanStack Query.
- Zustand.
- React Hook Form.
- Zod.
- Tailwind CSS.
- Radix UI.
- Lucide React.
- Recharts ou Apache ECharts.
- date-fns.
- Vitest.
- Testing Library.

- Playwright.

2.2 Firebase

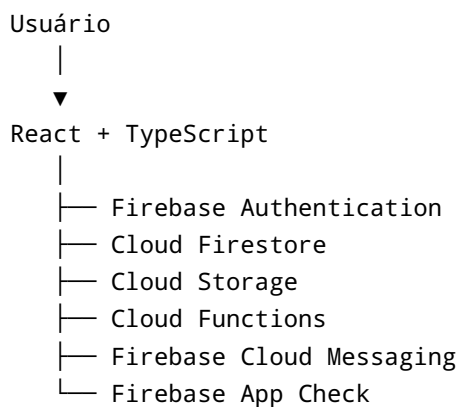
- Firebase Authentication.
- Cloud Firestore.
- Cloud Functions for Firebase.
- Cloud Storage for Firebase.
- Firebase Hosting.
- Firebase Cloud Messaging.
- Firebase App Check.
- Firebase Analytics, opcional.
- Firebase Performance Monitoring.
- Firebase Crashlytics para uma futura aplicação mobile.
- Firebase Local Emulator Suite.

2.3 Serviços complementares

Quando necessário:

- Cloud Scheduler.
- Cloud Tasks.
- Secret Manager.
- Google Cloud Logging.
- Google Cloud Monitoring.
- Serviço externo de envio de e-mail.
- Provedor de Open Finance.
- API de cotação de investimentos.

3. Arquitetura geral



3.1 Responsabilidade do frontend

O frontend será responsável por:

- Renderizar a interface.
- Gerenciar formulários.
- Fazer leituras permitidas no Firestore.
- Executar operações simples autorizadas pelas Security Rules.
- Manter estados locais da interface.
- Exibir relatórios e gráficos.
- Trabalhar com cache.
- Oferecer funcionamento offline em áreas apropriadas.
- Chamar Cloud Functions para operações críticas.

3.2 Responsabilidade das Cloud Functions

As Cloud Functions serão responsáveis por:

- Criar transferências financeiras.
- Processar pagamentos de fatura.
- Gerar parcelas.
- Gerar lançamentos recorrentes.
- Recalcular saldos consolidados.
- Importar arquivos.
- Detectar duplicidades.
- Processar convites.
- Gerenciar permissões.
- Enviar notificações.
- Integrar serviços externos.
- Atualizar cotações.
- Executar rotinas programadas.
- Processar operações que alterem vários documentos.
- Registrar auditoria confiável.

3.3 Comunicação com o Firebase

O frontend poderá acessar diretamente o Firestore para:

- Consultas autorizadas.
- Criação de lançamentos simples.
- Alteração de preferências.
- Leitura de contas, categorias e relatórios.
- Atualização de dados que não dependam de cálculos críticos.

Operações financeiras compostas deverão utilizar Callable Functions ou funções HTTP.

Exemplos:

```
Criar transferência  
Gerar compra parcelada
```

```
Pagar fatura
Importar extrato
Convidar participante
Recalcular saldo
Excluir grupo financeiro
```

4. Autenticação

O Firebase Authentication será utilizado para:

- Cadastro com e-mail e senha.
- Login com e-mail e senha.
- Login com Google.
- Recuperação de senha.
- Verificação de e-mail.
- Encerramento de sessão.
- Atualização de senha.
- Exclusão de usuário.
- Autenticação multifator, quando habilitada.

Cada usuário autenticado terá um identificador único:

```
request.auth.uid
```

Esse identificador deverá ser utilizado como referência principal do usuário em todos os documentos.

4.1 Documento do usuário

```
/users/{userId}
```

Exemplo:

```
{
  "displayName": "Lucas Oliveira",
  "email": "lucas@email.com",
  "photoURL": null,
  "currency": "BRL",
  "locale": "pt-BR",
  "timezone": "America/Sao_Paulo",
  "emailVerified": true,
  "onboardingCompleted": true,
  "createdAt": "Timestamp",
  "updatedAt": "Timestamp"
}
```

Não deverão ser armazenados no Firestore:

- Senha.
- Hash de senha.
- Tokens de autenticação.
- Credenciais bancárias.
- Código de segurança de cartão.

Essas responsabilidades pertencem ao Firebase Authentication ou ao provedor externo utilizado.

5. Organização dos dados no Firestore

O Cloud Firestore é um banco NoSQL orientado a documentos. Portanto, a aplicação utilizará coleções, documentos e subcoleções em vez de tabelas relacionais.

5.1 Estrutura principal

```
/users/{userId}

/financialGroups/{groupId}
  /members/{userId}
  /accounts/{accountId}
  /cards/{cardId}
  /invoices/{invoiceId}
  /transactions/{transactionId}
  /categories/{categoryId}
  /budgets/{budgetId}
  /recurrences/{recurrenceId}
  /goals/{goalId}
  /debts/{debtId}
  /investments/{investmentId}
  /assets/{assetId}
  /subscriptions/{subscriptionId}
  /imports/{importId}
  /rules/{ruleId}
  /notifications/{notificationId}
  /monthlySummaries/{summaryId}
  /auditLogs/{auditLogId}

/invitations/{invitationId}
```

Todos os dados financeiros deverão pertencer a um grupo financeiro.

Mesmo um usuário individual terá um grupo próprio.

6. Grupo financeiro

Caminho

```
/financialGroups/{groupId}
```

Documento

```
{
  "name": "Finanças da Família",
  "type": "family",
  "currency": "BRL",
  "ownerId": "user-id",
  "memberIds": [
    "user-id-1",
    "user-id-2"
  ],
  "createdAt": "Timestamp",
  "updatedAt": "Timestamp"
}
```

O campo `memberIds` poderá ser mantido no documento para facilitar consultas e regras de acesso.

Tipos

- `individual`
- `couple`
- `family`

Regras

- Todo grupo deverá possuir pelo menos um administrador.
 - Somente administradores poderão convidar ou remover participantes.
 - Um usuário poderá participar de mais de um grupo.
 - Um usuário poderá definir um grupo padrão.
 - A remoção de um participante não deverá excluir seus lançamentos antigos.
 - O grupo não deverá ser apagado enquanto houver operações críticas em processamento.
-

7. Participantes e permissões

Caminho

```
/financialGroups/{groupId}/members/{userId}
```

Documento

```
{
  "userId": "user-id",
  "displayName": "Lucas Oliveira",
  "email": "lucas@email.com",
  "role": "admin",
  "status": "active",
  "permissions": {
    "viewSharedTransactions": true,
    "createTransactions": true,
    "editOwnTransactions": true,
    "editAllTransactions": true,
    "manageAccounts": true,
    "manageCards": true,
    "manageBudgets": true,
    "manageMembers": true,
    "viewReports": true
  },
  "joinedAt": "Timestamp"
}
```

Papéis iniciais

- owner
- admin
- member
- viewer

Permissões sensíveis deverão ser validadas tanto nas Security Rules quanto nas Cloud Functions.

8. Contas financeiras

Caminho

```
/financialGroups/{groupId}/accounts/{accountId}
```

Documento

```
{
  "name": "Conta Corrente",
  "institutionName": "Banco do Brasil",
  "type": "checking",
  "ownerId": "user-id",
  "color": "#2563EB",
  "icon": "landmark",
  "currency": "BRL",
  "initialBalance": 250000,
  "currentBalance": 485000,
  "projectedBalance": 563000,
  "creditLimit": 0,
  "includeInAvailableBalance": true,
  "includeInNetWorth": true,
  "status": "active",
  "createdAt": "Timestamp",
  "updatedAt": "Timestamp"
}
```

Valores monetários

Os valores deverão ser armazenados em centavos, utilizando números inteiros.

Exemplo:

```
R$ 1.250,90 → 125090
```

Isso evita erros de ponto flutuante.

No frontend:

```
const amountInCents = Math.round(amount * 100)
```

Para apresentação:

```
const formatted = new Intl.NumberFormat('pt-BR', {
  style: 'currency',
  currency: 'BRL',
}).format(amountInCents / 100)
```

Tipos de conta

- `checking`

- savings
- cash
- digital_wallet
- payment
- investment
- joint
- other

Regra de saldo

O saldo não deverá ser recalculado no navegador varrendo todo o extrato.

O documento da conta deverá manter saldos consolidados:

- currentBalance
- projectedBalance

Esses valores deverão ser atualizados por Cloud Functions utilizando transações do Firestore.

9. Lançamentos financeiros

Caminho

```
/financialGroups/{groupId}/transactions/{transactionId}
```

Documento

```
{
  "type": "expense",
  "description": "Supermercado Extra",
  "amount": 15680,
  "competenceDate": "Timestamp",
  "paymentDate": "Timestamp",
  "dueDate": "Timestamp",
  "status": "confirmed",
  "accountId": "account-id",
  "cardId": null,
  "invoiceId": null,
  "categoryId": "category-id",
  "subcategoryId": "subcategory-id",
  "ownerId": "user-id",
  "createdBy": "user-id",
  "visibility": "shared",
  "paymentMethod": "debit_card",
  "merchant": "Extra",
  "tags": [
```

```
    "mercado"  
  ],  
  "notes": null,  
  "recurrenceId": null,  
  "installmentPlanId": null,  
  "installmentNumber": null,  
  "installmentTotal": null,  
  "transferId": null,  
  "attachmentIds": [],  
  "createdAt": "Timestamp",  
  "updatedAt": "Timestamp"  
}
```

Tipos

- income
- expense
- transfer_in
- transfer_out
- balance_adjustment
- card_purchase
- invoice_payment
- refund
- investment_contribution
- investment_withdrawal

Situações

- planned
- pending
- confirmed
- overdue
- cancelled
- refunded
- partially_refunded

Visibilidade

- private
- shared

Lançamentos privados deverão ser visíveis apenas para:

- O criador.
- O proprietário do lançamento.
- Administradores autorizados, caso a política do produto permita.

10. Transferências

Uma transferência deverá gerar dois lançamentos vinculados:

Conta de origem → transfer_out
Conta de destino → transfer_in

Ambos deverão compartilhar:

transferId

Exemplo:

```
{
  "transferId": "transfer-uuid",
  "type": "transfer_out",
  "accountId": "conta-origem",
  "destinationAccountId": "conta-destino",
  "amount": 50000
}
```

A transferência deverá ser criada exclusivamente por Cloud Function.

A função deverá:

1. Validar o usuário.
2. Validar acesso às duas contas.
3. Validar o valor.
4. Criar os dois lançamentos.
5. Atualizar os saldos das contas.
6. Registrar a auditoria.
7. Concluir tudo em uma transação atômica.

Transferências não deverão ser contabilizadas como receita ou despesa nos relatórios.

11. Categorias

Caminho

/financialGroups/{groupId}/categories/{categoryId}

Documento

```
{
  "name": "Alimentação",
  "type": "expense",
  "parentId": null,
  "icon": "utensils",
  "color": "#F97316",
  "essential": true,
  "fixed": false,
  "status": "active",
  "order": 10,
  "createdAt": "Timestamp",
  "updatedAt": "Timestamp"
}
```

Subcategorias também serão documentos da coleção `categories`, mas com o campo `parentId`.

Exemplo:

```
{
  "name": "Supermercado",
  "parentId": "alimentacao-category-id"
}
```

Categorias utilizadas em lançamentos não deverão ser apagadas definitivamente. Elas serão arquivadas com:

```
{
  "status": "archived"
}
```

12. Cartões de crédito

Caminho

```
/financialGroups/{groupId}/cards/{cardId}
```

Documento

```
{
  "name": "Nubank",

```

```
"institutionName": "Nubank",
"brand": "Mastercard",
"lastFourDigits": "1234",
"ownerId": "user-id",
"creditLimit": 1000000,
"availableLimit": 735000,
"closingDay": 3,
"dueDay": 10,
"paymentAccountId": "account-id",
"color": "#820AD1",
"status": "active",
"createdAt": "Timestamp",
"updatedAt": "Timestamp"
}
```

Segurança

Nunca deverão ser armazenados:

- Número completo do cartão.
- CVV.
- Senha.
- Data de validade completa, salvo necessidade real e tratamento adequado.

Para identificação, bastam:

- Nome.
- Instituição.
- Bandeira.
- Quatro últimos dígitos.

13. Faturas

Caminho

```
/financialGroups/{groupId}/invoices/{invoiceId}
```

Documento

```
{
  "cardId": "card-id",
  "referenceMonth": "2026-07",
  "closingDate": "Timestamp",
  "dueDate": "Timestamp",
  "status": "open",
  "totalAmount": 125000,
}
```

```
"paidAmount": 0,  
"remainingAmount": 125000,  
"purchaseCount": 8,  
"createdAt": "Timestamp",  
"updatedAt": "Timestamp"  
}
```

Situações

- future
- open
- closed
- partially_paid
- paid
- overdue
- cancelled

Regras

- Uma compra deverá estar vinculada a uma fatura.
- O fechamento deverá impedir alterações indevidas no ciclo.
- O pagamento não deverá ser contabilizado novamente como despesa.
- A despesa será reconhecida na compra.
- O pagamento será uma movimentação entre conta e obrigação do cartão.
- Estornos deverão alterar a compra e a fatura correspondente.
- A troca da data de vencimento não deverá alterar automaticamente faturas já fechadas.

14. Compras parceladas

Plano de parcelamento

```
/financialGroups/{groupId}/installmentPlans/{installmentPlanId}
```

```
{  
  "description": "Notebook",  
  "totalAmount": 600000,  
  "installmentCount": 12,  
  "installmentAmount": 50000,  
  "cardId": "card-id",  
  "categoryId": "category-id",  
  "purchaseDate": "Timestamp",  
  "status": "active",  
  "createdBy": "user-id",  
  "createdAt": "Timestamp"  
}
```

Cada parcela deverá ser registrada como um lançamento independente:

```
{
  "installmentPlanId": "installment-plan-id",
  "installmentNumber": 1,
  "installmentTotal": 12,
  "amount": 50000,
  "invoiceId": "invoice-id"
}
```

A geração das parcelas deverá ocorrer em Cloud Function.

15. Recorrências

Caminho

```
/financialGroups/{groupId}/recurrences/{recurrenceId}
```

Documento

```
{
  "type": "expense",
  "description": "Internet",
  "amount": 9990,
  "frequency": "monthly",
  "interval": 1,
  "startDate": "Timestamp",
  "endDate": null,
  "nextExecutionAt": "Timestamp",
  "accountId": "account-id",
  "categoryId": "category-id",
  "automaticConfirmation": false,
  "generateDaysBefore": 7,
  "status": "active",
  "createdBy": "user-id",
  "createdAt": "Timestamp",
  "updatedAt": "Timestamp"
}
```

Frequências

- daily
- weekly
- biweekly

- monthly
- bimonthly
- quarterly
- semiannual
- yearly
- custom

Uma Scheduled Function será executada periodicamente para:

1. Localizar recorrências próximas da execução.
2. Criar os lançamentos correspondentes.
3. Atualizar `nextExecutionAt`.
4. Evitar geração duplicada.
5. Registrar falhas.

Cada ocorrência deverá possuir uma chave única de controle:

```
recurrenceId + referenceDate
```

16. Orçamentos

Caminho

```
/financialGroups/{groupId}/budgets/{budgetId}
```

Documento

```
{
  "referenceMonth": "2026-07",
  "categoryId": "category-id",
  "plannedAmount": 120000,
  "spentAmount": 95580,
  "pendingAmount": 12500,
  "availableAmount": 24420,
  "usagePercentage": 79.65,
  "status": "warning",
  "createdAt": "Timestamp",
  "updatedAt": "Timestamp"
}
```

Estados

- healthy
- warning

- `reached`
- `exceeded`

Os campos calculados poderão ser mantidos no documento para reduzir leituras repetitivas:

- `spentAmount`
- `pendingAmount`
- `availableAmount`
- `usagePercentage`

Esses campos deverão ser atualizados por função sempre que um lançamento relacionado for criado, alterado, confirmado ou cancelado.

17. Resumos mensais

Para evitar que o dashboard consulte milhares de lançamentos, serão utilizados documentos agregados.

Caminho

```
/financialGroups/{groupId}/monthlySummaries/{referenceMonth}
```

Exemplo

```
{
  "referenceMonth": "2026-07",
  "income": 825000,
  "expenses": 531000,
  "result": 294000,
  "confirmedIncome": 825000,
  "confirmedExpenses": 481000,
  "pendingIncome": 0,
  "pendingExpenses": 50000,
  "availableBalance": 1258000,
  "projectedBalance": 1323000,
  "savingsRate": 35.64,
  "categoryTotals": {
    "housing": 148080,
    "food": 95580,
    "transport": 79650
  },
  "updatedAt": "Timestamp"
}
```

O dashboard deverá consultar principalmente:

- `monthlySummaries`.
- `accounts`.
- Fatura atual.
- Próximos vencimentos.
- Metas principais.
- Últimos lançamentos.

Isso reduz custos e melhora o desempenho.

18. Metas financeiras

Caminho

```
/financialGroups/{groupId}/goals/{goalId}
```

Documento

```
{
  "name": "Reserva de emergência",
  "targetAmount": 2000000,
  "currentAmount": 760000,
  "targetDate": "Timestamp",
  "monthlyContribution": 100000,
  "priority": "high",
  "linkedAccountId": "account-id",
  "progressPercentage": 38,
  "status": "active",
  "createdAt": "Timestamp",
  "updatedAt": "Timestamp"
}
```

Aportes deverão ser registrados como lançamentos vinculados à meta.

O `currentAmount` poderá ser atualizado por função para evitar recalcular todos os aportes em cada abertura.

19. Dívidas

Caminho

```
/financialGroups/{groupId}/debts/{debtId}
```

Documento

```
{
  "name": "Financiamento do veículo",
  "creditor": "Banco",
  "originalAmount": 6000000,
  "remainingBalance": 4200000,
  "interestRate": 1.49,
  "interestPeriod": "monthly",
  "installmentCount": 48,
  "paidInstallments": 12,
  "installmentAmount": 172500,
  "startDate": "Timestamp",
  "estimatedEndDate": "Timestamp",
  "status": "active",
  "createdAt": "Timestamp",
  "updatedAt": "Timestamp"
}
```

O pagamento de parcelas deverá:

- Criar uma despesa.
- Atualizar a quantidade de parcelas pagas.
- Atualizar o saldo da dívida.
- Registrar juros e amortização, quando informados.
- Atualizar o resumo patrimonial.

20. Patrimônio e investimentos

Investimentos

```
/financialGroups/{groupId}/investments/{investmentId}
```

```
{
  "name": "Tesouro Selic",
  "type": "fixed_income",
  "institutionName": "Corretora",
}
```

```
"quantity": 5.25,  
"averagePrice": 145000,  
"investedAmount": 761250,  
"currentValue": 795000,  
"liquidity": "D+1",  
"lastPriceUpdateAt": "Timestamp",  
"status": "active"  
}
```

Outros ativos

```
/financialGroups/{groupId}/assets/{assetId}
```

```
{  
  "name": "Renault Clio",  
  "type": "vehicle",  
  "purchaseValue": 2500000,  
  "estimatedValue": 2200000,  
  "includeInNetWorth": true,  
  "lastValuationAt": "Timestamp",  
  "status": "active"  
}
```

O patrimônio líquido deverá ser pré-calculado:

```
Ativos financeiros  
+ bens  
+ investimentos  
- dívidas  
= patrimônio líquido
```

O histórico mensal poderá ser armazenado em:

```
/financialGroups/{groupId}/netWorthHistory/{referenceMonth}
```

21. Assinaturas

Caminho

```
/financialGroups/{groupId}/subscriptions/{subscriptionId}
```

Documento

```
{
  "name": "Netflix",
  "amount": 4590,
  "frequency": "monthly",
  "nextBillingDate": "Timestamp",
  "paymentSourceType": "card",
  "paymentSourceId": "card-id",
  "categoryId": "category-id",
  "automaticRenewal": true,
  "status": "active",
  "createdAt": "Timestamp"
}
```

A assinatura poderá estar vinculada a uma recorrência.

22. Importação de extratos

Caminho

```
/financialGroups/{groupId}/imports/{importId}
```

Documento

```
{
  "fileName": "extrato-julho.ofx",
  "fileType": "ofx",
  "storagePath": "groups/group-id/imports/import-id/file.ofx",
  "accountId": "account-id",
  "status": "processing",
  "totalRows": 0,
  "importedRows": 0,
  "ignoredRows": 0,
  "duplicatedRows": 0,
  "errorRows": 0,
  "createdBy": "user-id",
  "createdAt": "Timestamp",
  "completedAt": null
}
```

Fluxo

1. O usuário envia o arquivo ao Cloud Storage.

2. Um documento de importação é criado.
3. Uma Cloud Function inicia o processamento.
4. Arquivos grandes podem ser enviados para Cloud Tasks.
5. O sistema interpreta os registros.
6. Possíveis duplicidades são identificadas.
7. O usuário confere uma prévia.
8. Os lançamentos são confirmados.
9. Os resumos e saldos são atualizados.

A importação deverá ser idempotente.

O sistema poderá gerar uma chave de deduplicação usando:

```
data + valor + descrição + conta + identificador bancário
```

23. Armazenamento de arquivos

O Cloud Storage será utilizado para:

- Comprovantes.
- Recibos.
- Boletos.
- Extratos.
- Faturas.
- Imagens de metas.
- Arquivos de importação.
- Exportações temporárias.

Organização

```
users/{userId}/profile/avatar.ext  
  
groups/{groupId}/transactions/{transactionId}/{fileId}.ext  
  
groups/{groupId}/imports/{importId}/original.ext  
  
groups/{groupId}/goals/{goalId}/{fileId}.ext
```

Regras

- O usuário deverá pertencer ao grupo.
- O tamanho deverá ser limitado.
- Apenas tipos permitidos deverão ser aceitos.
- O nome enviado pelo usuário não deverá ser utilizado diretamente como caminho.
- Arquivos privados não deverão possuir URL pública permanente.
- Metadados do arquivo deverão ser registrados no Firestore.

- Arquivos abandonados deverão ser removidos periodicamente.
-

24. Notificações

Firestore

```
/financialGroups/{groupId}/notifications/{notificationId}
```

Documento

```
{
  "userId": "user-id",
  "type": "budget_warning",
  "title": "Orçamento próximo do limite",
  "message": "Você já utilizou 88% do orçamento de Transporte.",
  "entityType": "budget",
  "entityId": "budget-id",
  "read": false,
  "createdAt": "Timestamp"
}
```

Canais

- Central interna.
- Firebase Cloud Messaging.
- E-mail, utilizando serviço externo.
- Push mobile em uma fase futura.

Eventos

- Conta próxima do vencimento.
- Conta atrasada.
- Fatura próxima do vencimento.
- Fatura próxima do fechamento.
- Orçamento próximo do limite.
- Orçamento ultrapassado.
- Saldo projetado negativo.
- Assinatura próxima da renovação.
- Meta atrasada.
- Falha na importação.
- Falha na integração bancária.

Scheduled Functions deverão identificar eventos futuros e gerar notificações.

25. Regras de categorização automática

Caminho

```
/financialGroups/{groupId}/rules/{ruleId}
```

Documento

```
{
  "name": "Categorizar Uber",
  "priority": 10,
  "conditions": [
    {
      "field": "description",
      "operator": "contains",
      "value": "UBER"
    }
  ],
  "actions": {
    "categoryId": "transport-category-id",
    "subcategoryId": "apps-category-id",
    "tags": [
      "transporte"
    ]
  },
  "status": "active",
  "createdAt": "Timestamp"
}
```

As regras deverão ser aplicadas:

- Durante importações.
- Durante sincronizações.
- Opcionalmente no cadastro manual.
- Em reprocessamentos solicitados pelo usuário.

Reprocessamentos em massa deverão utilizar Cloud Tasks.

26. Auditoria

Caminho

```
/financialGroups/{groupId}/auditLogs/{auditLogId}
```


Documento

```
{
  "actorId": "user-id",
  "action": "transaction.update",
  "entityType": "transaction",
  "entityId": "transaction-id",
  "before": {
    "amount": 15000
  },
  "after": {
    "amount": 18000
  },
  "source": "cloud-function",
  "createdAt": "Timestamp"
}
```

Registros de auditoria deverão ser criados por Cloud Functions para operações críticas.

O frontend não deverá ter permissão para editar ou apagar auditorias.

27. Security Rules

As regras deverão começar bloqueando tudo e liberar apenas operações explicitamente autorizadas.

Estrutura conceitual:

```
rules_version = '2';

service cloud.firestore {
  match /databases/{database}/documents {
    function isAuthenticated() {
      return request.auth != null;
    }

    function isGroupMember(groupId) {
      return isAuthenticated()
        && exists(
          /databases/{database}/documents/
            financialGroups/{groupId}/members/{request.auth.uid}
        );
    }

    function isGroupAdmin(groupId) {
      return isGroupMember(groupId)
        && get(
          /databases/{database}/documents/

```

```

        financialGroups/${groupId}/members/${request.auth.uid}
    ).data.role in ['owner', 'admin'];
}

match /users/{userId} {
    allow read, update: if isAuthenticated()
        && request.auth.uid == userId;

    allow create: if isAuthenticated()
        && request.auth.uid == userId;
}

match /financialGroups/{groupId} {
    allow read: if isGroupMember(groupId);
    allow create: if isAuthenticated();
    allow update: if isGroupAdmin(groupId);
    allow delete: if false;

    match /members/{memberId} {
        allow read: if isGroupMember(groupId);
        allow write: if isGroupAdmin(groupId);
    }

    match /transactions/{transactionId} {
        allow read: if isGroupMember(groupId);
        allow create: if isGroupMember(groupId);
        allow update: if isGroupMember(groupId);
        allow delete: if false;
    }

    match /auditLogs/{logId} {
        allow read: if isGroupAdmin(groupId);
        allow write: if false;
    }
}
}
}

```

As regras reais deverão também validar:

- Tipos dos campos.
- Valores obrigatórios.
- Valores monetários positivos.
- Campos que não podem ser alterados.
- Propriedade do lançamento.
- Visibilidade privada.
- Papéis e permissões.
- Tamanho de listas.
- Datas válidas.
- Status permitidos.

Security Rules não substituem validações de domínio nas Cloud Functions.

28. Firebase App Check

O App Check deverá ser habilitado para reduzir acessos indevidos aos recursos do Firebase.

Deverá proteger:

- Cloud Firestore.
- Cloud Storage.
- Callable Functions.
- Outros serviços compatíveis.

A implantação deverá ocorrer em duas etapas:

1. Monitorar requisições sem aplicar bloqueio.
2. Ativar enforcement após validar os ambientes legítimos.

O App Check complementa a autenticação, mas não substitui:

- Firebase Authentication.
 - Security Rules.
 - Validações do backend.
 - Controle de permissões.
-

29. Índices do Firestore

O Firestore exigirá índices para consultas compostas.

Índices prováveis:

```
transactions:
  groupId + competenceDate
  status + competenceDate
  accountId + competenceDate
  cardId + competenceDate
  invoiceId + competenceDate
  categoryId + competenceDate
  ownerId + competenceDate
  visibility + competenceDate

budgets:
  referenceMonth + categoryId

invoices:
  cardId + dueDate
```

```
cardId + status

notifications:
userId + read + createdAt

recurrences:
status + nextExecutionAt
```

Os índices deverão ser versionados em:

```
firestore.indexes.json
```

30. Estratégia de consultas

O frontend não deverá carregar coleções inteiras.

Todas as listas deverão utilizar:

- Filtros.
- Ordenação.
- Limite.
- Cursor.
- Paginação.

Exemplo conceitual:

```
query(
  collection(db, `financialGroups/${groupId}/transactions`),
  where('competenceDate', '>=', startDate),
  where('competenceDate', '<=', endDate),
  orderBy('competenceDate', 'desc'),
  limit(50),
)
```

A paginação deverá utilizar `startAfter`.

Não deverá ser usada paginação baseada em números de página e grandes deslocamentos.

31. Controle de custos

Como o Firestore cobra principalmente por operações, armazenamento e transferência, a aplicação deverá ser planejada para evitar leituras desnecessárias.

Estratégias

- Utilizar documentos de resumo mensal.
- Não recalcular dashboards lendo todos os lançamentos.
- Pagar extratos.
- Evitar listeners em coleções grandes.
- Desligar listeners quando a tela não estiver ativa.
- Armazenar totais consolidados.
- Evitar documentos muito grandes.
- Não duplicar dados sem necessidade.
- Utilizar cache local.
- Carregar apenas o período selecionado.
- Usar Functions para agregações pesadas.
- Definir limites máximos nas consultas.
- Monitorar uso e configurar alertas de orçamento.

Informações que podem ser desnormalizadas

- Nome da categoria no lançamento.
- Nome da conta no lançamento.
- Nome do cartão no lançamento.
- Totais mensais.
- Total por categoria.
- Saldo atual.
- Saldo projetado.
- Percentual do orçamento.
- Progresso da meta.

A desnormalização deverá ser controlada para evitar inconsistências.

32. Cloud Functions

Estrutura sugerida

```
functions/  
├── src/  
│   ├── auth/  
│   │   ├── on-user-created.ts  
│   │   └── delete-user.ts  
│   ├── groups/  
│   │   ├── create-group.ts  
│   │   ├── invite-member.ts  
│   │   └── accept-invitation.ts  
│   ├── transactions/  
│   │   ├── create-transaction.ts  
│   │   ├── update-transaction.ts  
│   │   └── create-transfer.ts
```

```

|   |   └─ refund-transaction.ts
|   └─ cards/
|       └─ create-installment-purchase.ts
|       └─ close-invoice.ts
|       └─ pay-invoice.ts
|   └─ recurrences/
|       └─ generate-recurrences.ts
|   └─ imports/
|       └─ start-import.ts
|       └─ process-import.ts
|       └─ confirm-import.ts
|   └─ reports/
|       └─ rebuild-monthly-summary.ts
|       └─ export-report.ts
|   └─ notifications/
|       └─ check-due-dates.ts
|       └─ send-notification.ts
|   └─ shared/
|       └─ auth.ts
|       └─ permissions.ts
|       └─ money.ts
|       └─ errors.ts
|   └─ index.ts

```

Tipos de função

Callable Functions

Para operações chamadas diretamente pelo frontend:

- Criar transferência.
- Criar compra parcelada.
- Pagar fatura.
- Convidar usuário.
- Processar importação.
- Gerar relatório.

Firestore triggers

Para reagir a alterações:

- Atualizar resumos.
- Atualizar orçamento.
- Atualizar limite do cartão.
- Criar auditoria.
- Enviar notificação.

Triggers deverão ser idempotentes, pois uma execução poderá ser repetida.

Scheduled Functions

Para:

- Gerar recorrências.
- Marcar lançamentos atrasados.
- Fechar faturas.
- Verificar vencimentos.
- Atualizar investimentos.
- Remover arquivos órfãos.
- Gerar snapshots patrimoniais.

Task Queue Functions

Para:

- Importações extensas.
- Reprocessamento de categorias.
- Exportação de relatórios.
- Processamento de grande número de lançamentos.
- Integrações sujeitas a limite de requisições.

33. Estrutura do frontend

```
src/
├── app/
│   ├── router/
│   ├── providers/
│   ├── layouts/
│   └── guards/
├── components/
│   ├── ui/
│   └── shared/
├── features/
│   ├── auth/
│   ├── dashboard/
│   ├── accounts/
│   ├── transactions/
│   ├── categories/
│   ├── cards/
│   ├── invoices/
│   ├── budgets/
│   ├── recurrences/
│   ├── goals/
│   ├── debts/
│   ├── investments/
│   ├── subscriptions/
│   └── imports/
```

```
|   ├── reports/
|   ├── family/
|   └── settings/
└── firebase/
    ├── app.ts
    ├── auth.ts
    ├── firestore.ts
    ├── functions.ts
    ├── storage.ts
    └── messaging.ts
└── hooks/
└── lib/
└── schemas/
└── stores/
└── types/
└── utils/
```

34. Estado no frontend

A separação deverá ser:

Dados do Firestore	→ TanStack Query
Sessão Firebase	→ Auth Provider
Estado global da UI	→ Zustand
Estado do formulário	→ React Hook Form
Validação	→ Zod
Filtros compartilháveis	→ URL
Estado local	→ useState

O Zustand não deverá duplicar coleções completas do Firestore.

35. Firebase Hosting

Como o frontend será uma SPA construída com React e Vite, deverá ser hospedado no Firebase Hosting.

Configuração esperada:

```
{
  "hosting": {
    "public": "dist",
    "ignore": [
      "firebase.json",
      "**/*.*",
    ],
  },
}
```



```

    "**/node_modules/**"
  ],
  "rewrites": [
    {
      "source": "**",
      "destination": "/index.html"
    }
  ]
}
}

```

O Firebase Hosting deverá fornecer:

- HTTPS.
- CDN.
- Domínio padrão.
- Domínio personalizado.
- Preview channels.
- Rollback.
- Integração com CI/CD.

Para essa aplicação SPA, o Firebase Hosting tradicional é suficiente. Firebase App Hosting seria mais relevante caso o projeto futuramente adotasse um framework full-stack com renderização no servidor.

36. Ambientes

Deverão existir projetos Firebase separados:

```

finance-app-dev
finance-app-staging
finance-app-prod

```

Não utilizar o mesmo projeto para desenvolvimento e produção.

Cada ambiente deverá possuir:

- Authentication próprio.
- Firestore próprio.
- Storage próprio.
- Functions próprias.
- Configurações próprias.
- Variáveis e secrets próprios.

Aliases:

```

{
  "projects": {

```

```
    "default": "finance-app-dev",  
    "staging": "finance-app-staging",  
    "production": "finance-app-prod"  
  }  
}
```

37. Desenvolvimento local

A Firebase Local Emulator Suite deverá ser utilizada para:

- Authentication.
- Firestore.
- Functions.
- Storage.
- Hosting.

Serviços externos deverão ser simulados durante testes automatizados.

O desenvolvimento local não deverá acessar dados reais de produção.

38. Testes

Frontend

- Vitest.
- Testing Library.
- MSW ou mocks da camada de repositório.
- Playwright.

Firebase

- Testes de Security Rules.
- Testes das Cloud Functions.
- Testes de transações.
- Testes com os emuladores.
- Testes de idempotência.
- Testes de permissões por papel.

Casos obrigatórios

- Usuário não autenticado não acessa dados.
- Usuário não acessa outro grupo.
- Participante comum não altera permissões.
- Transferência atualiza duas contas.
- Compra parcelada gera todas as parcelas.
- Pagamento de fatura não duplica despesa.

- Recorrência não gera duas ocorrências iguais.
 - Importação não duplica lançamentos.
 - Exclusão de categoria não quebra histórico.
 - Edição de lançamento atualiza orçamento e resumo.
 - Lançamento privado não é exibido para outro participante.
-

39. CI/CD

Pull request

Executar:

- Instalação de dependências.
- Typecheck.
- ESLint.
- Testes unitários.
- Testes de Security Rules.
- Testes das Functions.
- Build.
- Preview do Firebase Hosting.

Branch principal

Executar:

- Todos os testes.
- Build de produção.
- Deploy do Firestore Rules.
- Deploy dos índices.
- Deploy do Storage Rules.
- Deploy das Functions.
- Deploy do Hosting.

Deploys de produção deverão utilizar credenciais seguras e ambientes protegidos no GitHub.

40. Roadmap de desenvolvimento

Fase 1 — MVP individual

- Firebase Authentication.
- Onboarding.
- Grupo financeiro individual.
- Contas.
- Categorias.
- Receitas.
- Despesas.

- Transferências.
- Extrato.
- Dashboard mensal.
- Resumo mensal.
- Orçamento básico.
- Recorrências.
- Firebase Hosting.
- Security Rules.
- App Check.
- Exportação CSV.

Fase 2 — Cartões

- Cadastro de cartões.
- Faturas.
- Compras.
- Parcelamentos.
- Estornos.
- Pagamento de fatura.
- Calendário financeiro.
- Notificações.
- Projeção de saldo.

Fase 3 — Importação e automação

- CSV.
- OFX.
- Storage.
- Cloud Tasks.
- Detecção de duplicidades.
- Categorização automática.
- Regras personalizadas.
- Exportação XLSX e PDF.

Fase 4 — Uso familiar

- Grupos de casal e família.
- Convites.
- Papéis e permissões.
- Lançamentos privados.
- Despesas compartilhadas.
- Divisão proporcional.
- Reembolsos.
- Acerto entre participantes.

Fase 5 — Patrimônio

- Metas.
- Dívidas.
- Assinaturas.
- Investimentos.

- Bens.
- Histórico patrimonial.
- Integração com cotações.

Fase 6 — Open Finance

- Integração com provedor autorizado.
- Consentimentos.
- Sincronização bancária.
- Renovação de consentimento.
- Histórico de sincronizações.
- Tratamento de falhas.
- Conciliação bancária.

41. Escopo recomendado da primeira entrega

A primeira versão deverá conter:

1. Cadastro e login.
2. Criação automática do grupo individual.
3. Cadastro de contas.
4. Cadastro de categorias.
5. Receitas e despesas.
6. Transferências.
7. Recorrências mensais.
8. Extrato com filtros.
9. Dashboard mensal.
10. Orçamento por categoria.
11. Resumos mensais.
12. Exportação CSV.
13. Tema claro e escuro.
14. Responsividade.
15. Firestore Security Rules.
16. App Check.
17. Testes com emuladores.
18. Deploy no Firebase Hosting.

Cartões deverão ser implementados somente depois que contas, lançamentos, transferências e resumos estiverem estáveis.

42. Decisões técnicas principais

Utilizar Cloud Firestore

Motivos:

- Integração direta com o Firebase.
- Sincronização em tempo real.
- Suporte offline.
- Escalabilidade gerenciada.
- Security Rules.
- Boa integração com aplicações web e mobile.

Não utilizar Realtime Database inicialmente

O Cloud Firestore é mais adequado para:

- Consultas por período.
- Filtros compostos.
- Documentos organizados por domínio.
- Paginação.
- Estrutura financeira da aplicação.

Não depender exclusivamente do frontend

Operações financeiras críticas deverão passar por Cloud Functions para evitar:

- Manipulação de saldo.
- Transferências incompletas.
- Faturas inconsistentes.
- Parcelas faltantes.
- Alteração indevida de permissões.
- Auditorias falsas.

Manter valores em centavos

Todos os valores monetários deverão ser inteiros.

Manter documentos agregados

Dashboards e relatórios frequentes deverão utilizar resumos pré-calculados.

Tratar o Firestore como NoSQL

Não tentar reproduzir diretamente um banco relacional.

A modelagem deverá favorecer:

- Consultas conhecidas.
 - Dados agregados.
 - Desnormalização controlada.
 - Poucas leituras.
 - Documentos pequenos.
 - Operações idempotentes.
-

43. Critérios de aceite técnicos

A primeira versão será considerada pronta quando:

- Nenhum usuário conseguir acessar outro grupo financeiro.
- Os saldos permanecerem consistentes após criação, edição e cancelamento de lançamentos.
- Transferências forem atômicas.
- O dashboard não precisar ler todo o extrato.
- Recorrências não forem duplicadas.
- Valores monetários não apresentarem erros de precisão.
- Security Rules estiverem cobertas por testes.
- O aplicativo funcionar com os emuladores.
- O deploy for automatizado.
- O frontend funcionar adequadamente em desktop e celular.
- O projeto possuir ambientes separados.
- O usuário conseguir exportar seus dados.
- Logs e falhas de Functions puderem ser acompanhados.